Specification

Technical Field

The present invention belongs to the technical fields of distributed ledgers, smart contracts, multi-agent collaborative cooperation and decentralized settlement. Specifically, it relates to a multi-level automatic revenue allocation method and system based on dependent graph construction with a hard maximum depth limit of **18 derivation generations**, incremental state variable calculation, asymmetric gradient proportional cascading, dynamic weight allocation via fingerprint self-consistency similarity of multi-parent nodes, and status redirection transfer upon copyright expiration.

Background Art

With the popularization of multi-agent collaborative networks, users can package, encapsulate and release their own cognitive assets (such as methodologies, rule datasets, etc.). When downstream users or agents invoke these assets, or even conduct secondary development, inheritance and derivation based on them, existing clearing and settlement systems suffer from the following critical defects:

1. Unbounded revenue splitting and poor system efficiency: Most traditional multi-level distribution or revenue sharing systems adopt unlimited-generation splitting or tree recursive splitting. When a multi-agent collaboration chain is extremely long, a single payment transaction will trigger hundreds of on-chain token transfers, resulting in extremely high token transfer fee consumption and ledger confirmation latency, which severely reduces system efficiency or even causes system stagnation.

2. Unclear ownership for multi-parent inheritance and gray arbitrage: When a derived asset references or inherits multiple predecessor parent assets simultaneously, traditional settlement contracts cannot automatically distribute royalties based on the actual contribution of inherited assets in a trustless and tamper-proof manner during execution. Malicious downstream developers can easily conduct improper arbitrage through simple splicing and false original claims.

3. Lack of controllable end-stage backfeed mechanism for copyright protection terms: Existing digital copyright contracts either charge high commissions permanently, causing bloated ecological benefits and hindering knowledge dissemination; or lack term constraints, failing to automatically redirect revenue into a public inclusive backfeed pool upon expiration of a set protection period (e.g., **20 years**), without reasonable lifecycle status management.

4. Confusion between business models and technical implementations: When applying for multi-level revenue clearing solutions to patent offices, conventional schemes tend to focus purely on financial fund transfers and business rule definitions, lacking deep coupling with underlying ledger storage structures, Gas calculation ceilings, atomic reentrancy protection and other hard technical primitives, which are easily rejected as "mental activity rules or business methods".

Therefore, designing a multi-level automatic revenue allocation system deeply coupled with the underlying storage structure of decentralized distributed ledgers, while balancing Gas overhead ceilings and security protection against reentrancy attacks, is an urgent technical challenge in the fields of decentralized applications (DApps) and AI right confirmation.

Summary of the Invention

Technical Problems to Be Solved

To overcome excessive execution overhead caused by traditional multi-generation revenue sharing schemes, plagiarism risks arising from ambiguous multi-parent ownership, redundant ecological benefits brought by permanent royalties, and the tendency of conventional schemes to fall under the rejection threshold of business methods, the present invention proposes an intelligent automatic settlement and allocation scheme deployed on decentralized ledger state trees: "lineage depth truncation, incremental state variable clearing, similarity weighted distribution, and post-expiration backfeed to public pools".

Technical Solutions

To solve the above technical problems, the present invention provides a system for automatic multi-level revenue allocation of cognitive assets based on smart contracts.

Aspect 1: A system for automatic multi-level revenue allocation of cognitive assets based on smart contracts deployed on a distributed incremental right confirmation ledger, the system comprising:

1. A **dependency graph tree construction unit**: configured to, upon triggering of a sub-protocol node registration transaction, read parent protocol hash fingerprints declared in the data structure of the sub-protocol node, recursively construct a tree-shaped derived topology graph along version dependency records, and set a hard **maximum dependency generation limit threshold**; when the calculated recursive path depth exceeds the limit

threshold, an exception is automatically thrown and decentralized consensus chaining of the registration transaction is blocked;

2. A **state variable incremental calculation unit based on cascaded topology paths**: configured to, upon receiving a data packet of a commercial settlement event, intercept an asset liquidation request containing a transaction amount **R**, split the amount **R** into a first proportional value retained by the current node, a second proportional value allocated to the parent protocol layer, and a third proportional value allocated to the grandparent protocol layer satisfying a preset asymmetric descending gradient relationship, and write each proportional value into corresponding **pending withdrawal balance state mapping variables** of contracts in the ledger state tree in parallel as non-immediate transfer increments;

3. A **dynamic weight allocation module** deployed inside the state variable incremental calculation unit: configured to, when the sub-protocol node has multi-parent inheritance, read self-consistency similarity between feature vector fingerprints of each parent protocol, proportionally calculate allocation weights, and accordingly dynamically adjust pending withdrawal balance increments of corresponding parent creator accounts in the state variables;

4. A **lifecycle state transition contract module**: configured to store a creation timestamp of each protocol node in the ledger state tree; when the state variable incremental calculation unit allocates pending split amounts to a target node, if the difference between the current consensus timestamp and the creation timestamp of the node exceeds a set protection period time threshold, or the node is determined as a **root-level meta-protocol with zero private revenue**, a contract state machine transition is triggered to forcibly redirect and accumulate the split amount corresponding to the node into a pending withdrawal balance variable of an **ecological directional backfeed pool contract account**;

5. A **secure pull-type withdrawal unit**: configured to respond to a transfer transaction initiated by a payee, verify pending withdrawal balances under the initiator's name in the state variables, clear the pending withdrawal balance state variable, and then trigger an underlying on-chain transfer instruction to execute physical transfer to the payee, so as to physically isolate and eliminate reentrancy calls.

Preferably, the hard maximum dependency generation limit threshold set in the dependency graph tree construction unit is fixed to **18 generations**. If the recursive depth between the sub-protocol

node and the root-level meta-protocol with zero private revenue exceeds 18 generations, the consensus forcibly rejects chaining registration.

Preferably, in the state variable incremental calculation unit based on cascaded topology paths, the split proportion gradients corresponding to the first proportional value, the second proportional value and the third proportional value are hard-coded and locked as follows: **75% retained by the creator of the current node, 10% allocated to the parent protocol layer, and 15% allocated to the grandparent protocol layer**. In liquidation along the same cascaded path, the third proportional value allocated to the traced grandparent protocol layer is permanently higher than the second proportional value allocated to the parent protocol layer, so as to drive the proliferation of underlying public infrastructure via reverse benefit gravity.

Aspect 2: A method for automatic multi-level revenue allocation of cognitive assets based on smart contracts, executed by a computer processor, comprising the following steps:

1. Topology graph registration and verification: upon triggering of a sub-protocol node registration transaction, read parent protocol hash fingerprints declared in the data structure of the sub-protocol node, recursively construct a tree-shaped derived topology graph along version dependency records, and set a hard maximum dependency generation limit threshold; when the calculated recursive path depth exceeds the limit threshold, an exception is automatically thrown and decentralized consensus chaining of the registration transaction is blocked;
2. Fixed-proportion asymmetric cascaded clearing and bookkeeping: upon receiving a data packet of a commercial settlement event, intercept an asset liquidation request containing a transaction amount **R**, split the amount **R** into a first proportional value retained by the current node, a second proportional value allocated to the parent protocol layer, and a third proportional value allocated to the grandparent protocol layer satisfying a preset asymmetric descending gradient relationship, and write each proportional value into corresponding pending withdrawal balance state mapping variables of contracts in the ledger state tree in parallel as non-immediate transfer increments;
3. Dynamic weight allocation: when the sub-protocol node has multi-parent inheritance, read self-consistency similarity between feature vector fingerprints of each parent protocol, proportionally calculate allocation weights, and accordingly dynamically adjust pending withdrawal balance increments of corresponding parent creator accounts in the state variables;

4. Expiry judgment and redirection bookkeeping: store a creation timestamp of each protocol node in the ledger state tree; when allocating pending split amounts to a target node, if the difference between the current consensus timestamp and the creation timestamp of the node exceeds a set protection period time threshold, or the node is determined as a root-level meta-protocol with zero private revenue, trigger a contract state machine transition to forcibly redirect and accumulate the split amount corresponding to the node into a pending withdrawal balance variable of an ecological directional backfeed pool contract account;
5. External pull-type physical withdrawal: respond to a transfer transaction initiated by a payee, verify pending withdrawal balances under the initiator's name in the state variables, clear the pending withdrawal balance state variable, and then trigger an underlying on-chain transfer instruction to execute physical transfer to the payee, so as to physically isolate and eliminate reentrancy calls.

Preferably, if a sub-protocol node inherits multiple parent protocols {F_1, F_2, ..., F_n}, the smart contract reads fingerprint similarity feature scores S_i between feature vector fingerprints of each registered parent protocol and the sub-protocol node in the ledger runtime state, and calculates allocation weights via the formula below:

```
Weight_i = (S_i) / (∑_j=1^n S_j)
```

The smart contract dynamically adjusts pending withdrawal balance increments of corresponding parent creator accounts in the state variables according to the allocation weights; if there is no similarity difference, the amount at this layer is split equally.

Preferably, in the step of external pull-type physical withdrawal, the contract conducts fund transfer via a strict **two-phase atomic sequence**: at the entry of the withdrawal transaction, the pending withdrawal balance state variable of the caller is forcibly reset to zero within the consensus execution context to complete state tree update, and then an underlying on-chain transfer instruction is invoked to execute token transfer to the external wallet of the caller; if the transfer instruction returns an execution failure signal, a contract transaction rollback is triggered to restore the pending withdrawal balance state variable, so as to physically isolate concurrent reentrancy execution paths of external reentrancy calls by priority state modification.

Aspect 3: A ledger contract-side operation method in the method for automatic multi-level revenue allocation of cognitive assets based on smart contracts, executed by a smart contract clearing module deployed on a decentralized ledger node, comprising the following steps:

1. Lineage depth registration access control: upon receiving a sub-protocol node registration transaction, read parent protocol hash fingerprints declared in its data structure, recursively construct a derived topology graph along version dependency records; if the recursive path depth exceeds the limit threshold of **18 generations**, reject registration and trigger blocking of consensus chaining;
2. Transaction balance cascaded increment calculation: upon receiving an asset liquidation request containing a transaction amount **R**, split the amount **R** into a first proportional value retained by the current node, a second proportional value allocated to the parent protocol layer, and a third proportional value allocated to the grandparent protocol layer satisfying an asymmetric descending gradient relationship, and write each proportional value into pending withdrawal balance state mapping variables of the ledger state tree in parallel as increments;
3. Multi-parent similarity weighting and redirection: if multiple parent protocols are inherited, read their fingerprint similarity feature scores to calculate allocation weights and dynamically update corresponding pending withdrawal balance increments; if the difference between the creation timestamp of a target node and the current consensus timestamp reaches **20 years**, or the node is a root-level meta-protocol with zero private revenue, trigger state transition to redirect and accumulate the corresponding split amount into a pending withdrawal balance variable of the public backfeed pool contract account.

Preferably, the step of transaction balance cascaded increment calculation further includes a timing sequence verification sub-step: read timestamps in the liquidation request and verify timing sequence rationality to defend against replay attacks targeting the asset liquidation request; in multi-thread concurrent processing of ledger nodes, add concurrent control state identifiers based on timestamps and owner public keys to pending withdrawal balance state mapping variables corresponding to protocol nodes.

Aspect 4: A creator-side revenue extraction method in the method for automatic multi-level revenue allocation of cognitive assets based on smart contracts, executed by a creator payment terminal device, comprising the following steps:

1. Account pending balance query: send a pending withdrawal balance query request from a local terminal to the multi-level allocation smart contract deployed on the decentralized ledger, to obtain values of pending withdrawal balance state mapping variables accumulated via cascaded allocation under the current account;

2. Pull-type withdrawal transaction signature initiation: use the creator's asymmetric private key to digitally sign a withdrawal request containing the withdrawal amount and payment account, and initiate a withdrawal transaction to the smart contract, so as to trigger deduction and anti-reentrancy transfer verification based on a two-phase atomic sequence at the smart contract side;
3. Clearing and settlement fund receipt: after the smart contract side passes verification, clears and updates the pending withdrawal balance state mapping variable, and invokes an underlying transfer instruction to complete physical transfer, receive the asset balance transferred on-chain at the local terminal.

Preferably, in the step of account pending balance query, the pending withdrawal balance query request is sent to the smart contract via an asymmetric encrypted channel, and returned values of pending withdrawal balance state mapping variables are protected via public key encryption; in the step of pull-type withdrawal transaction signature initiation, mix the latest block hash provided by the decentralized ledger or a one-time random **Nonce** during digital signature, to physically block third-party interception of withdrawal requests for withdrawal replay attacks.

Aspect 5: A computer-readable storage medium storing a computer program thereon, wherein the program, when executed by a processor, implements the method according to any one of Claims 4 to 10.

Beneficial Effects

The beneficial effects brought by the present invention include:

1. Controllable settlement overhead and anti-network deadlock: Through the hard limit of **18 inheritance generations**, potential unbounded recursive on-chain computation is eliminated at the protocol registration stage. Meanwhile, the pull-type withdrawal design converts real-time multi-path token transfers at settlement into lightweight state variable incremental updates, reducing the computational complexity of multi-path revenue allocation from O(N) to O(1), and fundamentally preventing risks of virtual machine Gas exhaustion or ledger concurrent transaction deadlock.
2. Scientific multi-parent right confirmation: A dynamic allocation formula based on similarity is introduced to automatically distribute revenue according to fingerprint coincidence ratios, protecting reasonable property rights of underlying inheritors and curbing infringing arbitrage behaviors such as splicing and plagiarism.
3. Sustainable ecological circulation mechanism and anti-monopoly defense line: The 20-year

protection period state machine and zero-revenue design for meta-protocols enable core underlying infrastructure protocols to automatically redirect backfeed to the community while satisfying property right incentives, promoting secondary reproduction of knowledge, and rendering the patent resistant to invalidation requests for "abusing technology to implement monopolies".

4. Pure software distribution and medium-level protection: An independent claim for computer-readable storage media is added, covering contract deployment packages, revenue allocation SDKs and terminal extraction programs encapsulating the automatic allocation and withdrawal verification program within the patent's direct protection scope, preventing competing products from evading tort liability through pure software code distribution.

Brief Description of the Drawings

Figure 1: Schematic diagram of the core system physical topology and clearing-settlement relationship of the present invention;

Figure 2: Timing diagram of multi-parent asymmetric three-tier cascaded clearing bill flow based on fingerprint similarity;

Figure 3: Lifecycle state transition flow chart for copyright expiration and meta-protocol revenue redirection.

Detailed Description of the Embodiments

5.0 Terminology Definition and Generalization of Sovereign Nouns

This section establishes consistent definitions for core assets and technical concepts of the present invention:

1. Meta-Cognitive Protocol: Represents underlying meta-rules and root protocol standards, defining cascading thresholds and allocation constants for revenue allocation logic, functionally corresponding to a standardized protocol parsing and decision center.

2. **Value Protocol**: Standardized protocol data format containing allocation parameters, parent node fingerprints and author addresses, functionally corresponding to a value protocol encapsulation container.

3. **Decentralized Distributed Right Confirmation Ledger**: Also referred to as a "distributed incremental right confirmation ledger", representing a ledger operating environment carrying execution of revenue allocation smart contracts and fund evidence storage, functionally corresponding to a distributed settlement and clearing network.

As an ecological integration description of the automatic clearing and allocation system, the present system can be seamlessly deployed in a three-layer separated multi-agent system architecture composed of a **Rule Cognition Layer**, an **Isolated Asset Layer** and a **Computing Execution Layer** (as disclosed in the related parent case patent of the present invention "A System and Method for Operating Multi-Agent Systems Based on Three-Layer Separation"), serving as its underlying settlement and backfeed mechanism. The automatic allocation system physically runs on the distributed incremental right confirmation ledger; asset data and node dependency relationships for clearing are directly derived from incremental hash fingerprint ownership registrations generated by local encrypted cognitive asset libraries, while specific clearing logic, allocation weights and lifecycle judgments are driven by execution results of exclusive mantras and self-consistency assertions declared within standardized cognitive protocol containers. The present invention realizes a physical closed-loop operation of cognitive assets throughout the full lifecycle of "right confirmation – isolated execution – security audit – automatic revenue allocation – ecological backfeed".

5.1 Dependency Graph Tracking Algorithm with a Limit of 18 Derivation Generations

When a sub-protocol node conducts registration on the distributed incremental right confirmation ledger, the smart contract must traverse all declared parent fingerprint hashes of the node and recursively trace upward along version dependency records to the top-level root protocol to construct a tree-shaped dependency graph.

Contract depth verification:

$$Depth(Node) \leq 18$$

If the path distance (i.e., recursive depth) between the sub-protocol node and the top-level root meta-protocol exceeds the hard limit of **18 generations**, the contract forcibly triggers an exception interrupt and blocks decentralized consensus chaining of the registration transaction. This design physically avoids consensus computation overflow and clearing latency caused by malicious construction of ultra-deep call chains.

5.2 State Variable Increment Calculation and Weight Allocation Algorithm Based on Cascaded Topology Paths

When a sub-protocol node is commercially invoked by downstream parties and generates transaction funds (intercepted transaction amount) **R**, the smart contract immediately intercepts the funds and performs parallel read-only extraction of corresponding parent and grandparent node information.

1. Asymmetric split structure and state variable writing:

To avoid frequent on-chain token transfers consuming excessive Gas, the contract does not execute instant token transfers, but performs bookkeeping via update of mapping variables in the decentralized ledger state tree.

The second proportional value allocated to the parent layer is fixed at 10% of transaction amount **R**.

The third proportional value allocated to the grandparent layer is fixed at 15% of transaction amount **R**.

This proportion is fixed, with the grandparent layer share permanently higher than the parent layer share, incentivizing top-level creators to support and foster lower-layer sub-protocols. The creator of the current node retains the first proportional value of 75% of funds. These three proportional values are written as increments into pending withdrawal balance state mapping variables of the contract:

```
Balances[CurrentNode] ← Balances[CurrentNode] + R × 75%
Balances[ParentNode] ← Balances[ParentNode] + R × 10%
Balances[GrandparentNode] ← Balances[GrandparentNode] + R × 15%
```

1. Multi-parent similarity weight:

   If a sub-protocol node inherits multiple parent protocols {F_1, F_2, ..., F_n}, the smart contract reads fingerprint similarity feature scores S_i between feature vector fingerprints of each registered parent protocol and the sub-protocol node in the runtime ledger state, and calculates allocation weights via the formula below:

```
Weight_i = (S_i) / (∑_j=1^n S_j)
```

The smart contract dynamically adjusts pending withdrawal balance increments of each parent node according to the allocation weights:

```
Balances[F_i] ← Balances[F_i] + R × 10% × Weight_i
```

If there is no similarity difference, the 10% quota at this layer is split equally.

During multi-thread concurrent processing of the settlement calculation by ledger nodes, the system adds optimistic lock identifiers to the pending withdrawal balance of the node to verify timestamps and version numbers. If concurrent competition conflicts are detected, a spin-retry

mechanism is adopted to guarantee atomicity of bookkeeping, physically preventing double-spending and state overwriting errors.

5.3 Pull-Type Withdrawal Mechanism for Reentrancy and Gas Overflow Prevention

To defend against reentrancy attacks or clearing failures caused by excessive token transfer consumption when there is a large number of parent nodes, the system adopts a two-stage pull-type withdrawal design.

1. Pending balance bookkeeping stage: As described above, only mapping variables in the distributed incremental right confirmation ledger are updated during clearing:

```
Balances[Recipient] ← Balances[Recipient] + Payout
```

1. Withdrawal transfer stage: The creator actively initiates a withdrawal transfer transaction. The contract conducts fund transfer via a strict two-phase atomic sequence: at the entry of the withdrawal transaction, the pending withdrawal balance state variable of the caller is forcibly reset to zero within the consensus execution context to complete state tree update, and then an underlying on-chain transfer instruction is invoked to execute token transfer to the external wallet of the caller; if the transfer instruction returns an execution failure signal, a contract transaction rollback is triggered to restore the pending withdrawal balance state variable, so as to physically isolate concurrent reentrancy execution paths of external reentrancy calls by priority state modification.

   This logic forcibly reduces the computational complexity of settlement transactions to a constant O(1), guaranteeing the security of clearing and settlement. When initiating queries and withdrawals, the creator terminal sends requests through an asymmetric encrypted channel and verifies signatures based on the latest block hash and one-time Nonce, preventing interception of communication packets and withdrawal replay attacks.

5.4 Expiry Status Transition and Zero-Revenue Backfeed of Meta-Protocols

1. 20-year protection period counter:

   Upon protocol creation, the contract records an absolute creation timestamp T_create. Every time split bookkeeping is executed, the contract dynamically calculates the lifecycle difference of the node. If the difference between the current timestamp and the creation timestamp is greater than or equal to 20 years (i.e., 20 × 365 × 24 × 3600 seconds), the node's split receiving status automatically transitions from Protected to Public.

2. Hard binding of ecological backfeed pool:

   Once a node enters the Public status, all split revenue generated by the node is redirected to an ecological directional backfeed pool (specific contract address, e.g., 0x000000000000000000000000000000000000fade). The ecological directional backfeed pool features a hard-coded address, immutability and public auditability. All accumulated funds can only be used for operation and maintenance of underlying meta-protocols and distribution subsidies of public ecological resources, ensuring purely public welfare attributes at the technical level, eliminating subjective judgment of pure commercial operation, and meeting the subject eligibility requirements of patent examination guidelines. Upon deployment of a meta-protocol, its creator address is hard-coded as the backfeed pool address without any private privileged revenue, ensuring that foundational protocols fully belong to the public ecosystem.

5.5 Vertical Industry Application Scenarios and Embodiments

The automatic allocation system can be widely deployed in multiple industries with urgent demands for copyright incentives and public backfeed. Typical deployment embodiments are listed below:

1. Industrial manufacturing scenario: In collaborative derivative design of process drawings and control rules within smart factories, the system automatically performs revenue allocation for commercial invocations of multi-level process derived protocols. The dependency graph construction unit traces multi-level process node dependencies (e.g., Process C references Process B, and B references Process A). Revenue clearing is automatically distributed to accounts of process inventors at all levels according to the asymmetric gradient (75%/10%/15%), and splits are automatically backfed to the factory's public technology upgrading fund pool upon expiration of the protection period. This scenario operates on a three-layer separated multi-agent system operating architecture (namely a Rule Cognition Layer, an Isolated Asset Layer and a Computing Execution Layer physically isolated from one another), and is implemented based on any one of the methods according to Claims 4 to 10 of the present invention.

2. Government confidential scenario: In settlement of derivative authorization of multi-department official document rules and compliance protocols within electronic government affairs, multi-departments derive sub-rules based on root-level government legal meta-rules. The system automatically tracks the topology of official document processing rules, and calculates incremental bookkeeping according to similarity weighted proportions and cascading depths when expense settlement is triggered by document circulation. When certain official document processing sub-

rules have a creation timestamp older than 20 years, the contract triggers state transition to forcibly redirect corresponding split amounts into a public pool for government information construction. This scenario operates on a three-layer separated multi-agent system operating architecture, and is implemented based on any one of the methods according to Claims 4 to 10 of the present invention.

3. Financial data scenario: In credit rating risk control reasoning with multi-model fusion within banks, algorithm rules of multi-model creators are encapsulated as protocol nodes. When a credit system invokes a model group for fused risk control reasoning, the system intercepts clearing requests, reads fingerprint similarity features of each model, calculates dynamic allocation weights according to the formula, and updates each proportion as parallel increments in bookkeeping. Pull-type two-stage processing and public key encryption are adopted during withdrawals to guarantee fund security. This scenario operates on a three-layer separated multi-agent system operating architecture, and is implemented based on any one of the methods according to Claims 4 to 10 of the present invention.

5.6 Description of Anti-Avoidance Feature Design

As a critical anti-avoidance feature design of the present invention, if the technical solution of "dependency graph depth truncation + incremental state variable clearing + secure pull-type withdrawal + post-expiration backfeed" disclosed in the present invention is not adopted during actual deployment, and traditional unlimited-generation splitting or instant token transfer schemes are directly used, the following technical and systemic defects will arise:

(1) In multi-agent collaborative clearing with long chains or complex topologies, Gas Limit Exceeded or unbounded recursive deadlock of consensus virtual machines will be triggered, paralyzing the entire decentralized settlement network;

(2) Direct physical transfers during reasoning invocation and clearing stages are highly vulnerable to concurrent reentrancy attacks of smart contracts, leading to malicious emptying of fund pools;

(3) Lack of a controllable end-stage backfeed redirection mechanism allows a small number of creators to permanently monopolize underlying meta-protocols of the ecosystem, ultimately resulting in solidification and bloating of the distributed development ecosystem.

Through organic combination of the above technical features, the present invention completely eliminates the above technical risks and alternative avoidance schemes, greatly improving the inventiveness and invalidation resistance of the patent.

5.7 Equivalent Replacement Closing Paragraph

Finally, a person of ordinary skill in the art can understand that core inventive concepts of the present invention such as "dependency graph tree construction", "state variable incremental calculation", "secure pull-type withdrawal" and "lifecycle state transition" are not limited only to the specific software and hardware structural forms listed in the present embodiments, but cover all equivalent replacements and logical variants foreseeable by a person skilled in the art without departing from the spirit and essence of the present invention, all of which shall fall within the protection scope of the present invention.